

| Backend Engineer

010-9101-5429 | digle117@gmail.com | github.com/PreAgile | astro-paper-23v.pages.dev

📄📄📄

📄·📄·📄 📄📄 📄·📄📄 📄 📄📄📄 📄📄, 📄📄📄 📄📄, 📄📄 📄, 📄 📄, 📄 📄 📄📄 📄📄 📄 5📄 📄📄 📄📄📄📄. 📄📄 📄📄 📄📄 SaaS 📄📄 📄📄 **API** 📄 **20** 📄, 📄📄 📄📄 **100** 📄, 📄 📄 **12** 📄 📄📄 📄📄 📄 📄📄📄 📄📄 📄, 📄 📄📄 📄📄 ADR 📄 PR 📄📄 📄📄 📄📄 📄📄📄.

NestJS 📄 📄 📄📄📄 📄📄 📄 📄, 📄📄📄, 📄📄 DB 📄 📄 📄, 📄 📄📄, 📄 📄📄 📄 📄📄

Kotlin/Spring/JPA/Kafka/Resilience4j 📄📄📄 📄📄📄 JVM 📄📄 📄📄 📄📄📄 📄📄. 📄·📄📄📄📄📄 📄📄 📄📄 📄📄 📄 📄📄 📄📄. **Kotlin** 📄📄 📄📄📄 **Kotest** 📄 **organization** 📄 📄📄 **6** **PR** 📄 📄📄, **LINE Armeria** 📄 MDC export 📄 PR 📄, **Spring Batch** 📄 4📄 📄 📄📄📄 📄 chunk scanning 📄 📄 PR 📄 📄📄📄.

📄📄 📄📄 📄📄

📄📄	Before → After	📄📄
📄📄 📄 📄	📄 800 → 90 📄 (88.75% ↓)	📄 IP 📄 📄📄, 📄 📄📄
📄📄 📄📄	70% → 98%	📄📄 📄 📄 📄📄
📄📄 📄📄	99.2% (📄 6📄)	📄 📄 📄📄📄 FIFO + lease TTL
📄📄 📄📄 📄	0 / 6 📄	Idempotency-Key + 📄 📄 📄
Akamai 📄📄 📄📄	77.8% → 100% (18/18 iter)	Referrer Warming 📄 📄
📄📄 📄📄📄 webhook race	📄·📄📄 📄📄 📄	📄📄 📄 📄📄 📄 📄 📄📄
📄📄📄	Kotest 6 PR 📄 (org 📄) / Armeria 1 PR 📄 📄 / Spring Batch · Spring Cloud Gateway 📄 📄 + PR 📄 📄	GitHub PR 📄

📄📄📄 📄📄

JVM 📄📄 OSS 📄 📄📄 📄 — 📄 **6** · 📄 📄 **1** · 📄📄 📄 + **PR** 📄 📄.

Kotest — Kotlin 📄📄 📄📄📄 (4.7k stars)

2026.03 📄📄📄📄 sksamuel 📄 📄📄 📄📄 **organization** 📄 📄, 📄 📄 6 PR 📄.

- [#5905](https://github.com/kotest/kotest/pull/5905) — 📄📄 📄📄 Test Metadata Public API 📄📄 📄 (+355/-16, 5 📄 16h 📄 📄)
- [#5835](https://github.com/kotest/kotest/pull/5835) — 📄📄 data class 📄 📄 diff
- [#5807](https://github.com/kotest/kotest/pull/5807) — JSON Schema anyOf/oneOf 📄 📄

- [#5795](https://github.com/kotest/kotest/pull/5795) — JSON Matchers `Json` `Matchers` `Matchers`
- [#5789](https://github.com/kotest/kotest/pull/5789) — `@OnlyInputTypes` `Matchers` `Matchers` `Matchers`
- [#5756](https://github.com/kotest/kotest/pull/5756) — `shouldHaveSingleElement` `Matchers` `Matchers`

Armeria — LINE `Java` `RPC` `Framework` (5.1k stars)

- [#6683](https://github.com/line/armeria/pull/6683) — `RequestContextExporter` `BuiltInProperty` `Logback MDC export` `Logback MDC export` (+339/-5, `Logback MDC export` 1/2 `Logback MDC export`, `Logback MDC export` `Logback MDC export`)

Spring Batch — `Spring Batch` `PR` `PR` (2.9k stars)

- [#3946](https://github.com/spring-projects/spring-batch/issues/3946) — `fault-tolerant step chunk scanning` `Spring Batch` `Spring Batch` (documentation issue "documentation issue" `documentation issue`)
- [#4478](https://github.com/spring-projects/spring-batch/issues/4478) — `conditional flow decider order` `Spring Batch` (this is a bug "this is a bug" `Spring Batch`)

Spring Cloud Gateway — `Spring Cloud Gateway` `Spring Cloud Gateway` `Spring Cloud Gateway`

- [#3311](https://github.com/spring-cloud/spring-cloud-gateway/issues/3311) — `Apache HttpClient5` `Spring Cloud Gateway` `Spring Cloud Gateway` (documentation issue `Spring Cloud Gateway`, `Spring Cloud Gateway`, 5.1.x `Spring Cloud Gateway`)
- [#4024](https://github.com/spring-cloud/spring-cloud-gateway/issues/4024) — `API Versioning Predicates` `Spring Cloud Gateway` (가 `Spring Cloud Gateway` `Spring Cloud Gateway`)

📄 — 📄(Lemong) | | 📄 2024.11 ~ 📄

📄 📄

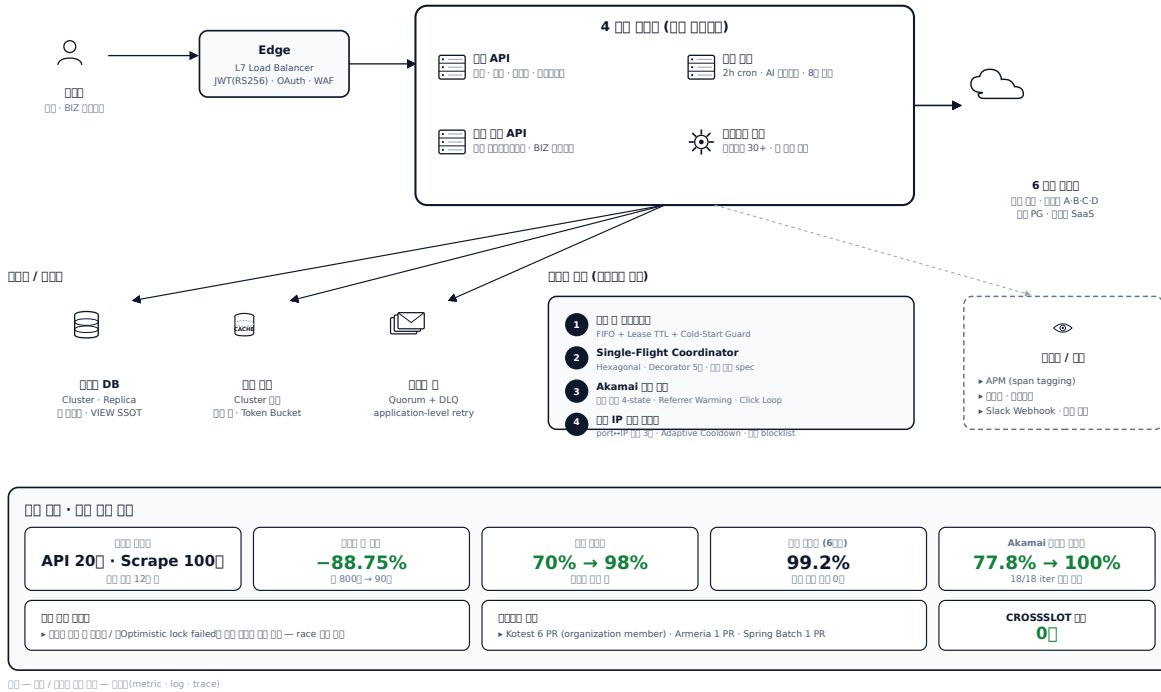
📄📄📄 📄📄📄 📄📄 📄 📄 📄 SaaS 📄 📄📄📄 📄📄📄. 📄📄 **API** 📄 **20** 📄, 📄📄 📄📄 **100** 📄, 📄 📄 **12** 📄 📄📄 📄📄, 📄📄 30+ worker 가 📄📄 📄📄📄. 📄📄 4 📄📄📄 📄📄📄 📄📄.

📄📄	📄📄 📄📄	📄📄
📄 API 📄📄	📄·📄·📄·📄📄 (BIZ 📄 ↔ 📄)	REST API 137 controllers / 📄📄 📄📄 152
📄 📄 📄📄	2📄 cron 📄 📄 · AI 📄📄 · Active-Active 8📄 📄	5,700+ 📄 / 13,000+ 📄
📄 📄 API	📄📄 📄 📄 · 📄📄 📄📄📄 · AI 📄 📄 · BIZ 📄📄	📄 📄📄📄 Hash + ZSet
📄📄 📄 📄📄	6 📄 📄📄 📄📄 · 📄 📄 📄📄 · Akamai 📄 · 📄 IP 📄	📄📄 worker 30+

📄 📄📄 📄📄 📄📄📄 · 📄📄 📄 (Quorum Queue) · 📄📄 📄 (Cluster 📄) · 📄📄 DB (Cluster) 가 📄📄 📄 📄📄 APM (span tagging) · 📄📄 📄📄 (가 📄📄) · Slack Webhook (📄 📄 · cron 📄/📄/📄) 📄 📄 📄📄 📄📄.

A1. 00 00 00 SaaS — 0000 00000

4 00 0000 · 6 00 0000 00 · 00 00000 · 00 00 0



A1. 00 0000 00000

0000 00·00(A) → 00 00 0000·00 0000(B) → 00 0000 00·00 00 00(C) → AI 000000·00 0000(D) 0 0000 15 0 0000 00 → 00 → 00 0000 000000.

0000 A. 00·00

00 1. 00 webhook 40 0000 — 00 0 (ABA-safe) · DB UNIQUE · 00000 · | 0 00 2025.09 ~ 00 0

00 — 00 PG webhook 0 00 00 00000 00 00000 0000 50 00 000000. ① PG 00 retry(5xx/timeout) ② 00 00 schedule 10 00 0 fallback retry 0 PAID/FAILED 가 00 00 00 ③ 00 00 0 0000 ④ 00 00 00 00 ⑤ Load Balancer keep-alive 0000. 00 INSERT 0 00 row 00 00 → 00 00 → 0000 20 00 → 00000 00 00 → 00 0 00 00000 00000 000000. 00 000000 000000 in-memory dedupe 가 가, ORM 0 findOne → save 0000 read-modify-write 0000 race window 가 000000. 00 00 0 000000 TTL 00 0 00 0000 00 00 0000 ABA 00 (Martin Kleppmann 0 Redlock 00000 0000 hazard) 가 0000 0 00, DB UNIQUE 000000 PG retry 0000 10000 0000 0 0000, 0000000000 00 INSERT race 0 00 0 00000.

00 — 00 00000 00 race 0 00 0 00 40 0000 0 00000 0000000.

1. 00 0 (ABA-safe) — 00 00 SETNX EX 30s + 00 lock value · Stripe Idempotency Key 00000 0 0000 lock value 0 ownership 00

2. DB UNIQUE — 00 0 imp_uid + merchant_uid + status · 1000 dedupe window · 00 00 fail-open 0000 00 00

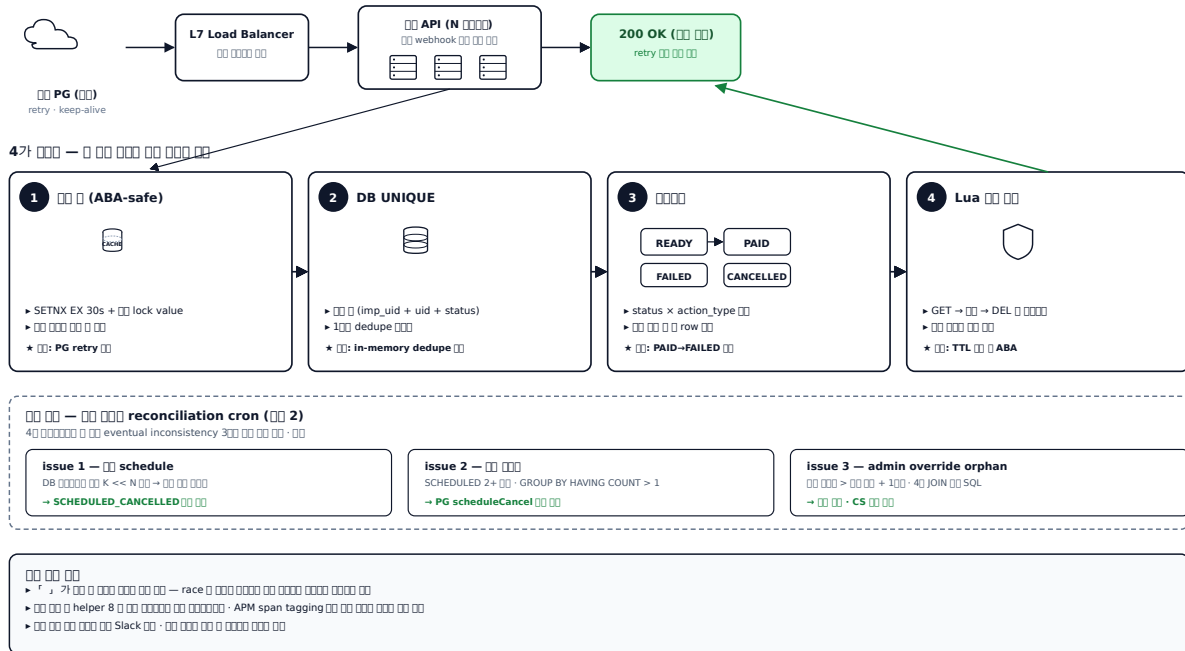
3. **상태** — READY → PAID / FAILED / CANCELLED × action_type **row** · **row** (PAID / FAILED / CANCELLED) **row** row **row**

4. **Lua** **row** — GET → **row** → DEL **row** **row** · **row** **row** **row** **row**, TTL **row** **row** ABA **row**

row SETNX + Lua DEL helper **row** **row** 8 **row** **row** (promotional upgrade · change plan · verify code · **row** **row**) **row** helper **row** **row** **row**.

A2. **webhook** 4 **row** — **row** · DB **row** · **row** · **row**

race · ABA · **row** · out-of-order **row** 4 hazard **row** **row** **row**



A2. **webhook** 4 **row** **row** **row**

row

- **row** **row** **row** **row** **row** **row** **row** **row** **row** **row** race 가 **row** **row** **row** **row** **row** **row** **row** **row**
- **row** **row** **row** **row** **row** Slack **row** **row** — **row** **row** **row** **row** **row**
- APM span tagging **row** **row** **row** **row** (amount_mismatch, coupon_select) **row** **row** **row**
- **row** **row** helper(delIfValueMatches)가 **row** **row** 8 **row** **row** **row** (promotional upgrade · change plan · verify code · **row** **row** **row**) **row** helper **row** **row** **row**

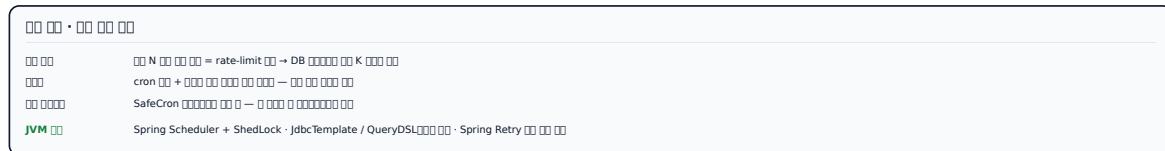
JVM/Spring **row** — Spring @Transactional(propagation/isolation **row**) · Redisson distributed lock + watchdog · DB unique constraint + JPA OptimisticLockException **row** · @RestControllerAdvice **row** ER_DUP_ENTRY **row**

00 — 00 1 0 40 000000 0 00 300 silent 000 000 00000.

- 이 때 DB가 읽기 / 쓰기 모드 모두 가능하도록 설정함. 이 때 PG API가 DB가 읽기 모드에 있는지 확인하고, DB가 Scheduled로 PG의 schedule_status가 eventual consistency로 설정함.

A3. $\mathbb{R}^n$ reconciliation — 3 inconsistency $\mathbb{R}^n$ · $\mathbb{R}^n$

CS 1000 DB 1000 K << N 4-way



11

- 5

- SQL admin override

JVM/Spring — Spring Scheduler + ShedLock (cron) · JdbcTemplate / QueryDSL · Spring Retry + RateLimiter · PG · ApplicationEventPublisher · Slack

3. 3월 3일 - 3월 3일 · schedule fallback 3월 3일 idempotency 2025.11 ~ 3월 3일

3월 3일 - 3월 3일 3월 3일 3월 3일. 3월 3일 schedule 3월 3일 3월 3일 amount 3월 3일 3월 3일 3월 3일 silent failure 가 가 3월 3일. schedule 3월 3일 3월 3일 A 3월 3일 50% 3월 3일 amount 3월 3일 3월 3일 3월 3일 A 가 3월 3일/3월 3일 3월 3일 PG 3월 3일 amount 3월 3일 3월 3일 → 3월 3일 3월 3일. 가 3월 3일 3월 3일 API 가 3월 3일 3월 3일 3월 3일 3월 3일 3월 3일 3월 3일 + 3월 3일 plan + 3월 3일 user 3월 3일 3월 3일 3월 3일 API 3월 3일 update 3월 3일 3월 3일 3월 3일 3월 3일.

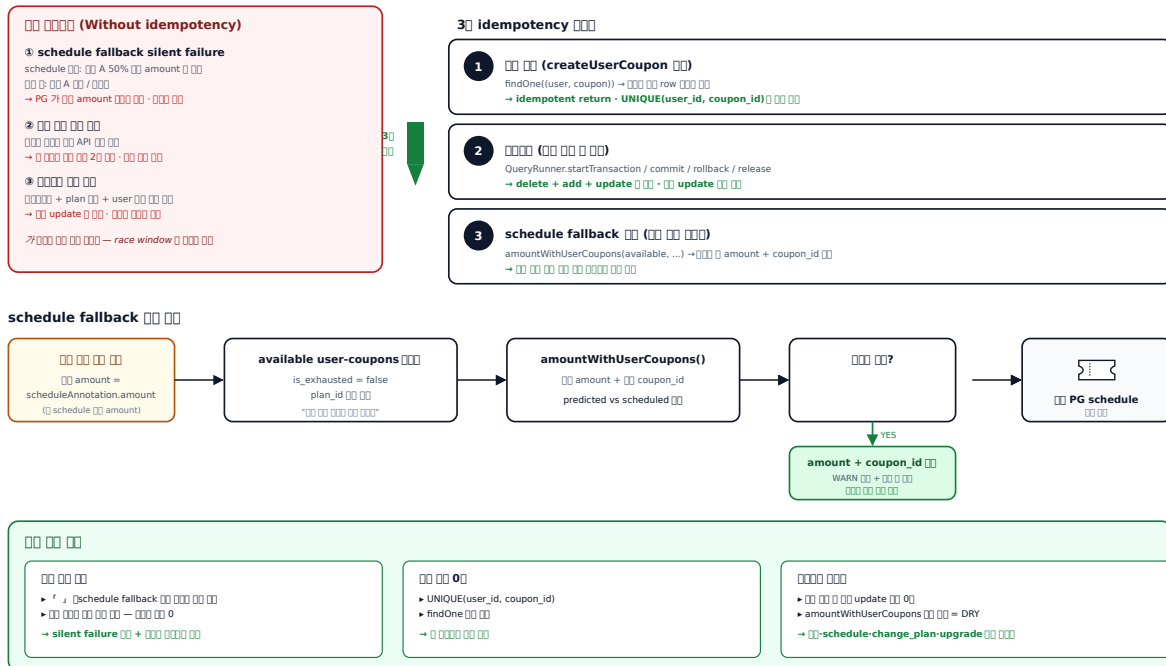
3월 3일 - 3월 3일 idempotency 3월 3일:

1. 3월 3일 - createUserCoupon 3월 3일 findOne((user, coupon)) 3월 3일 row 3월 3일, 3월 3일 3월 3일 (idempotent return)
2. 3월 3일 - ORM QueryRunner 3월 3일 3월 3일 (startTransaction / commit / rollback / release) - delete + add + update 3월 3일 3월 3일 3월 3일
3. **schedule fallback** 3월 3일 - schedule 가 3월 3일 3월 3일 user-coupons 3월 3일 3월 3일 3월 3일 3월 3일 → 3월 3일 3월 3일 amount + coupon_id 3월 3일

amountWithUserCoupons 3월 3일 3월 3일 · schedule · change_plan · promotional upgrade 3월 3일 3월 3일 3월 3일 DRY 3월 3일.

A4. 3월 3일 3월 3일 - 3월 3일 idempotency 3월 3일

3월 3일 3월 3일 · schedule fallback 3월 3일 - 3월 3일 3월 3일 · silent failure 3월 3일



A4. 3월 3일 3월 3일 idempotency 3월 3일

3월 3일

- 3월 3일 3월 3일 schedule fallback 3월 3일 3월 3일 3월 3일 3월 3일 3월 3일 3월 3일 3월 3일 3월 3일.

- `findByUserAndCouponId` — `UNIQUE(user_id, coupon_id) + findOne` 를 사용하여 중복을 방지.
- `update` 메서드에서 `update` 메서드를 사용.

JVM/Spring 관련 — JPA `@OneToMany` + `@Version` 사용 · Spring `TransactionTemplate` 사용 · Hibernate `@PostLoad` · `schedule fallback` 사용 · Bean Validation `@AssertTrue` · `amount` 가 0 이하

□□□ **B.** □□ □□ □□□ · □□ □□□ □□

4. 1,000+ row — X-lock + | 2025.12 ~

1,000+ row ~ 1 row = 1 row = 1~6 row. INSERT 1,000+ row (platform, platform_id) 가 findOne → save race row 가. ORM REPEATABLE READ + UNIQUE race window 가 ER_DUP_ENTRY 1,000+ row. 1,000+ row /가 row.

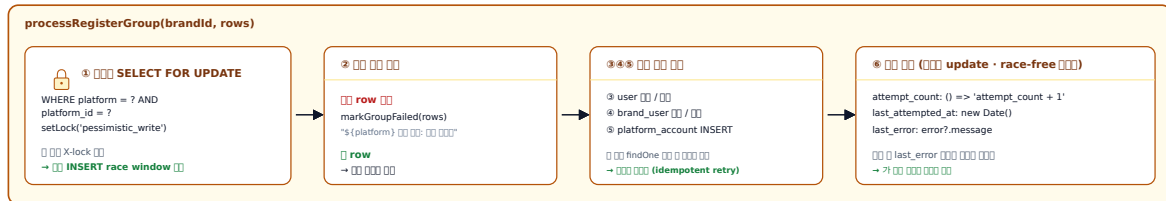
SELECT ... FOR UPDATE + groupKey = email + business_number @Transactional() = (platform, platform_id) X-lock marking. findOne + update(attempt_count + 1) race-free PENDING → VALIDATED → REGISTERING → REGISTERED | REGISTER_FAILED last_error · last_attempted_at · attempt_count row 가.

B4. 1,000+ row — SELECT FOR UPDATE + race window

1,000+ row = race window + row



processRegisterGroup(brandId, rows)



가 —



가

@Transactional + UNIQUE race window ER_DUP_ENTRY 가 markGroupFailed("\${platform} ")

가

1,000 → 950 row 50 row

가

1,000+ platform_id row JVM: JPA PESSIMISTIC_WRITE · REQUIRES_NEW

B4. 1,000+ row — X-lock +

1,000+ row /가 row

- platform_id row row 0.
- 1,000 950 row, 50 row 50 row —

JVM/Spring — JPA LockModeType.PESSIMISTIC_WRITE · Spring

@Transactional(propagation=REQUIRES_NEW) JpaSpecificationExecutor findAllById + saveAll batch.

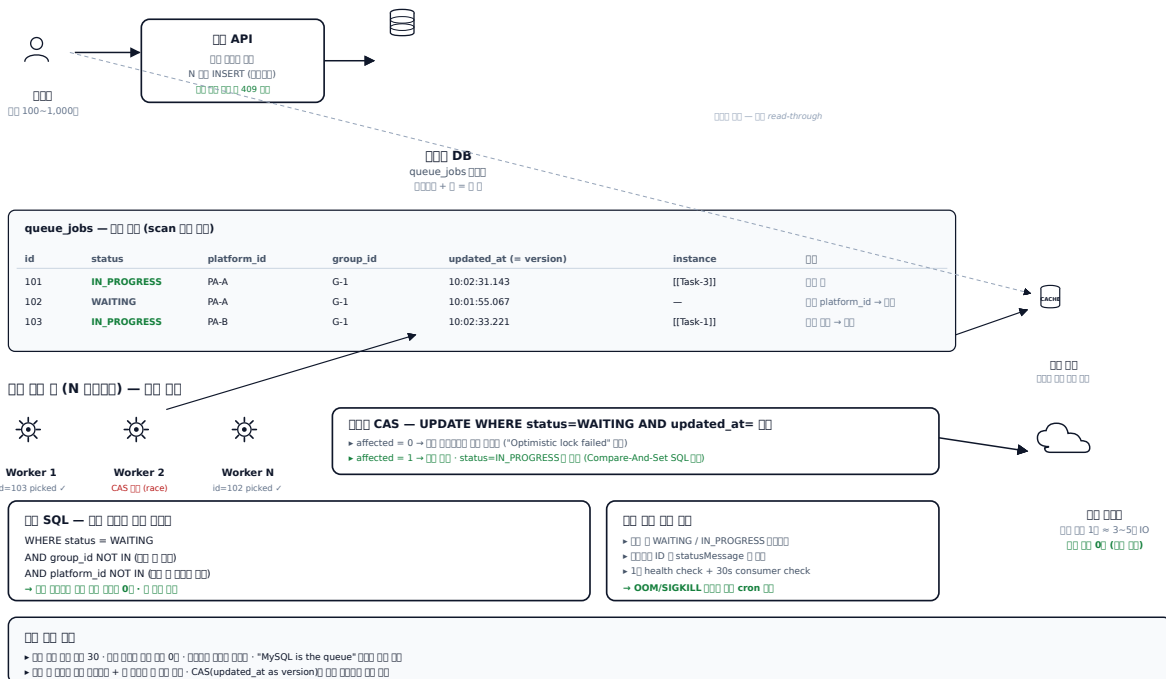
5. 100~1,000 IOPS DB CAS | 2025.10 ~

100~1,000 IOPS DB CAS | 2025.10 ~

DB ORM = CAS(Compare-And-Set) SQL updated_at version UPDATE WHERE id=? AND status=WAITING AND updated_at=affected=0

B1. DB CAS

MySQL is the queue, CAS



B1. RDS Queue + CAS

30

- 30, 100~1,000 IOPS DB CAS
- Job N modified by another instance (optimistic lock failed) race
- 0, 100~1,000 IOPS DB CAS
- 100~1,000 IOPS DB CAS

JVM/Spring — Spring Batch · JobRepository 가 · JPA @Version +
OptimisticLockException CAS · Coroutines + supervisorScope (Kotlin) @Async
ThreadPoolTaskExecutor (Java) · Micrometer .

7. Active-Active 8개 노드 배포 + | 배포 전략 2026.02

배포 — 배포할 노드 배포할 노드 배포할 노드 8개(blue/green/yellow/purple/orange/cyan/white/black) Active-Active 배포 전략. 8개 노드 배포 down/up 노드 traffic gap 노드 503 노드 배포 전략. 노드 배포 배포·노드 배포 배포 throw 가 노드, 노드 배포 노드 배포 노드 배포 노드 배포 노드 배포(가가 ≡ 노드 → 노드 노드 → 배포할 노드 노드 7개 노드 노드 → 노드 retry 노드 21개 노드)노드 노드 노드 배포.

배포 — 노드 배포 노드 배포 graceful drain 노드 **5-step** 배포 배포 + 배포 배포 **dict** 노드 노드 노드 배포. dynamic 노드 노드 → 노드 노드 배포(노드 노드, 배포할 노드 → 503 노드) → 8개 노드 down + 노드 up → health check(120 × 5s = 10min budget) → 노드 노드 + reload readiness 노드. **가** 배포 배포 배포 0(노드 노드 = 노드 노드 노드) 가 노드 노드 배포.

B5. Active-Active 8개 노드 배포 + 배포 전략 배포 전략

8개 노드 배포 배포 배포 배포 배포 배포 배포 · 노드 노드

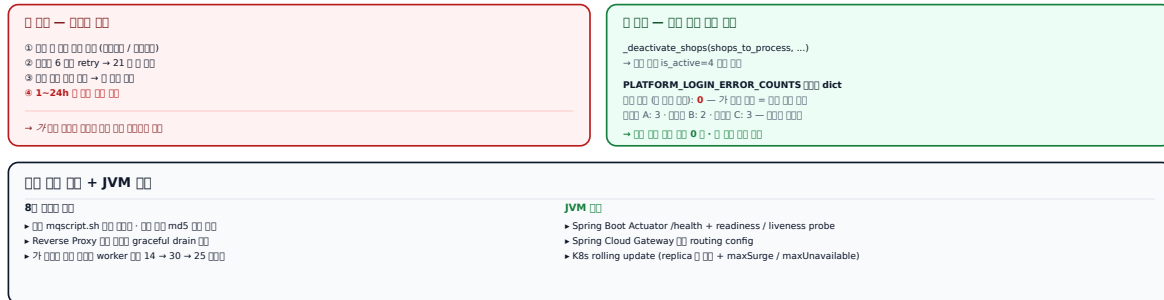
8개 노드 배포 (노드 배포)



배포 전략 5-step 배포 전략 (노드 mqscript.sh)



노드 배포 배포 — 배포 배포 dict



B5. Active-Active 8개 노드 배포 + 배포 전략 배포 전략

배포

- 8개 노드 배포 노드 배포 배포 배포 배포 배포 배포.
- 노드 배포 — 노드 배포 노드 배포 배포 배포, 가≡ 가 노드 0.
- 가** 배포 배포 worker 노드 14 → 30 → 25 배포.

JVM/Spring 배포 — Spring Boot Actuator /health + readinessProbe / livenessProbe · Spring Cloud Gateway 배포 Reverse Proxy 배포 dynamic config · K8s rolling update 가 배포 배포 배포

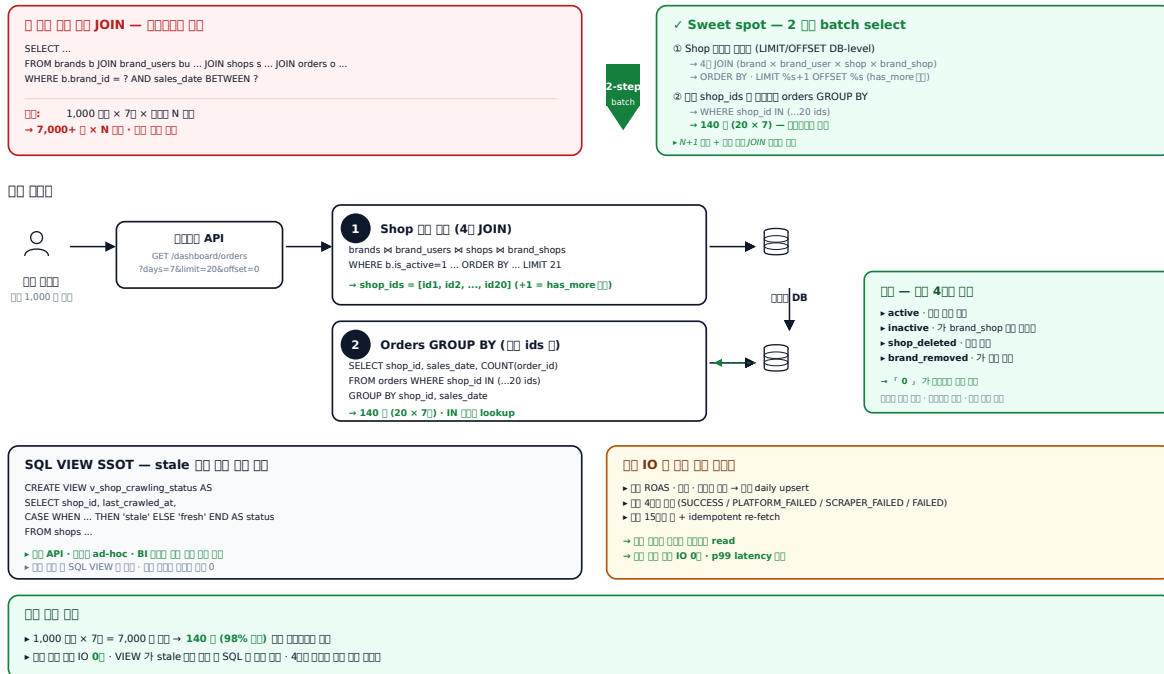
8. BIZ 1,000 × 7 - 2 + DB-level pagination + SQL VIEW SSOT 2026.01

— BIZ 1 1,000 7 ROAS / / , 7, (stale) , JOIN brand → ... → orders (1,000 × 7 × N) + IO ORM 4 + dynamic where + dynamic order by + dynamic LIMIT 가 .

— 「select shape N+1 JOIN sweet spot 2-batch select + SQL VIEW SSOT + IO . 1 shop (brand × brand_user × shop × brand_shop 4, LIMIT +1 has_more DB-level pagination), 2 ~20 shop_ids orders GROUP BY (140 = 20 × 7). 4(active / inactive / shop_deleted / brand_removed) 0 stale SQL VIEW API · ad-hoc · BI (single source of truth). daily upsert + 4 read.

B3. BIZ - 2 + SQL VIEW SSOT + IO

1 → 1,000 × 7 - N+1 + JOIN sweet spot



B3. BIZ 2 + SQL VIEW SSOT

- 1,000 × 7 2 + DB-level LIMIT/OFFSET IO 0.
- VIEW stale SQL .
- 150 + idempotent re-fetch .
- 「raw SQL 0 read.

JVM/Spring 2020 — Spring Data JPA + QueryDSL 2.0.0 / dynamic order by · Native Query 2020 batch select · Hibernate @FormulaJoinTable 2020 SQL VIEW 2020 (@Subselect) · @EntityGraph 2020 N+1 2020.

□□□ **C.** □□ □□□ □□ · □ □□ □□

9. 30+ worker race | 2025.11 ~

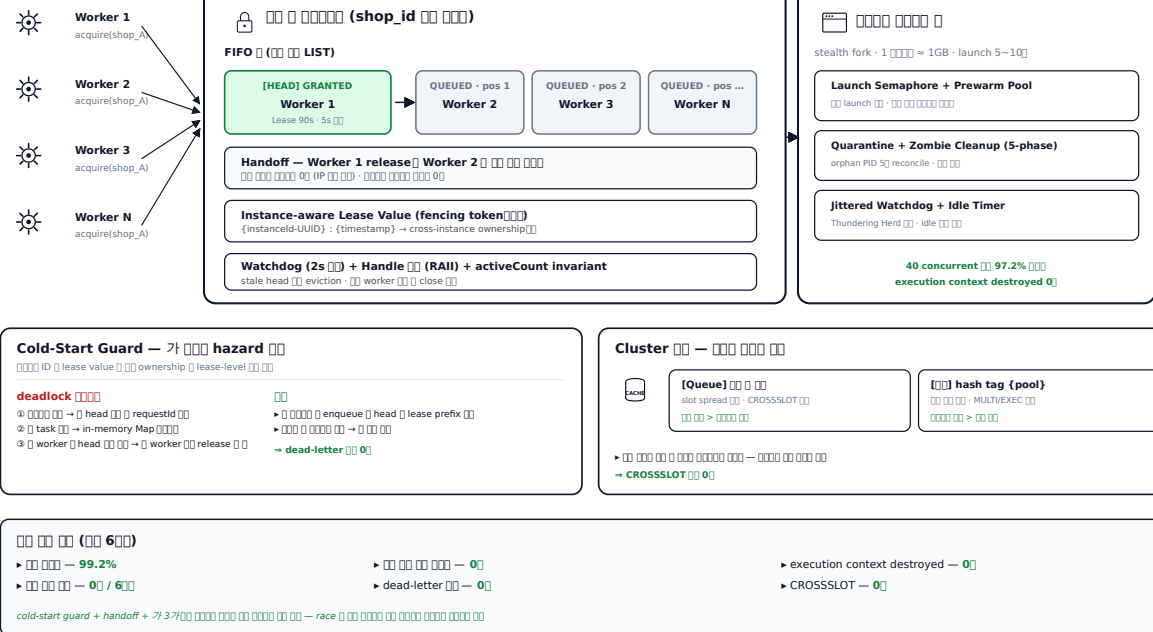
— 30+ consumer (shop_id) API(getReviews, addReply, getStores, keepAlive)가 30+ worker 가 30+ page navigation execution context destroyed, IP 1~24(1~24) mutex 1 1GB+, launch 5~10 → race N in-process Map Cluster CROSS SLOT hazard **cold-start deadlock** — in-memory Map head requestId, worker head requestId worker release

— **4 layer**: ① FIFO (LIST) ② per-request lease(90s TTL, 5s) ③ **instance-aware lease value** = {instanceId-UUID} : {timestamp} (Martin Kleppmann fencing token cross-instance ownership lease-level) ④ handoff pattern (release entry 가 가 cross-instance ownership lease-level) Cluster CROSS SLOT queue reputation(hash tag {pool}) attach/release Handle (RAII) activeCount invariant, pendingClose 5(immediate / defer / idle-timer / hasWaitingRequests handoff / release) worker close, release close. forceRelease(kill switch), forceTerminate(activeCount 0), evictStaleHead(2 head lease LREM) 3.

C1. 30+ worker race | 4 layer + Handoff + 3

30+ worker race | 4 layer + Handoff + 3

30+ worker race | 4 layer + Handoff + 3



`Queue`

- `Queue` `99.2%` (`6000`).
- `Queue` `0` / `6000` — `Queue` + `Idempotency-Key` `Queue`.
- `execution context destroyed` `0`.
- `Queue` `Queue` `dead-letter` `Queue` `0` — `cold-start guard` `Queue` `Queue`.
- `CROSSLOT` `0` — `hash tag` / `Queue` `Queue` `Queue` `Queue` `Queue` `Queue`.
- `Queue` `Queue` `Queue` `0`.

JVM/Spring `Queue` — `Redisson RFairLock (FIFO + lease auto-extend)` `Queue` `ShedLock` + `RLock` `Queue`
· `Kotlin Coroutines Mutex` + `withLock` `Queue` `in-process layer` · `@PreDestroy` `Queue` `graceful release` ·
`Micrometer gauge` `Queue` `Queue` `Queue`.

□□

- □ □□ **800** → **90** □ (**88.75%** □□) — □□ □□□ □□.
- □□ □□□ **70%** → **98%** — □□□ □□ □ □□ □□□□.
- □□ □□ SPOF □□(HA □□ □□ □□), □□□ □□□□ □□.
- □□ IP 가 legacy □□□ □□□□□ □□ **0**.
- CROSSLOT □□ **0** — hash tag {pool} □□ □.
- ASN □□ □□ + KR 가 + □□ □□ □□(failed outcome □□) □□.

JVM/Spring □□ — Spring Cloud Gateway + Resilience4j Retry / CircuitBreaker / Bulkhead □

□ · WebClient ExchangeFilterFunction □□ proxy resolver □□□ · Spring Configuration

@ConfigurationProperties + @Validated □ env □□ □□□.

11. Akamai Bot Manager — 4-state · Referrer Warming · Click Loop (77.8% → 100%) 2026.02

— Akamai Bot Manager 가 4-state 4 silent 99% stealth fork fingerprint sensor 가 가 401/403 pending 403 sensor → 403.

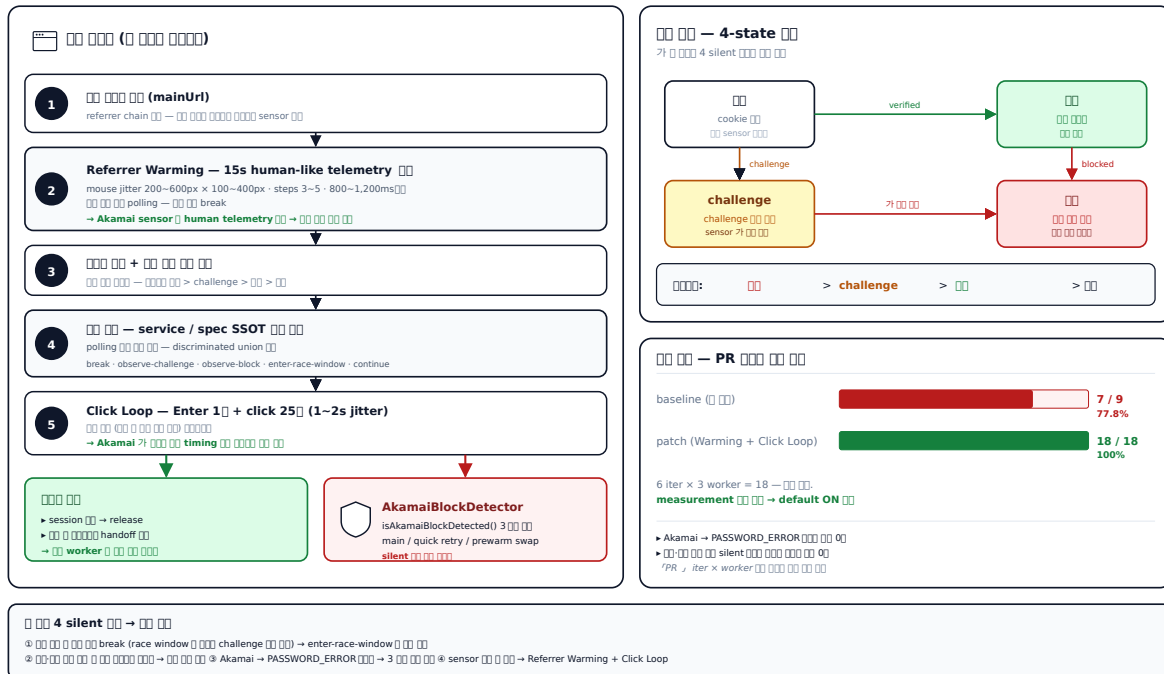
4 가 silent :

- 4-state break → race window challenge
- + 5 4-state
- Akamai application-level 401/403 password retry (Full Retry)
- sensor break → sensor 가 (1~2) silent

— 4-state 4 가 : (a) 4-state > challenge > > (b) service spec SSOT (discriminated union — break / observe-challenge / observe-block / enter-race-window / continue) + (c) Referrer Warming — URL referrer chain, 15 human-like mouse jitter(200~600px × 100~400px × steps 3~5 × 800~1,200ms) → sensor 가 human telemetry → + (d) Click Loop — submit Enter 1 → page click 25 (1~2 jitter) + (e) 3 — Akamai helper main loginResult + Quick Retry + Prewarm Swap 4-state silent 가 .

C3. Akamai Bot Manager — 4-state · Referrer Warming · Click Loop

4-state 4 silent 77.8% → 100% (18/18 iter)



C3. Akamai + 4-state

- 実行時間 **77.8% → 100%** (Referrer Warming 実行回数, 18/18 iter, 6 iter × 3 worker, default ON 実行).
- Akamai 実行 → PASSWORD_ERROR 実行回数 **0**.
- 実行回数 実行回数 silent 「 」 実行回数 **0**.
- PR 実行回数 iter × worker 実行回数 実行回数 実行回数 実行回数 (7/9 → 18/18).

JVM/Spring 実行 — Spring @Component 実行回数 実行回数 (sealed interface) · WebClient interceptor 実行回数 実行回数 実行回数 · Resilience4j TimeLimiter + Retry 実行 · Kotlin sealed class PollingAction 実行 discriminated union.

12. Single-Flight Coordinator — Hexagonal + Decorator 5, 7 spec | 2026.01

— platformId) N (single-flight · coalescing).

- Map 80+ / /
- invariant** — cause / (forceRelease record finally cleanup record) / throw (sync throw Map throw)
- spec /

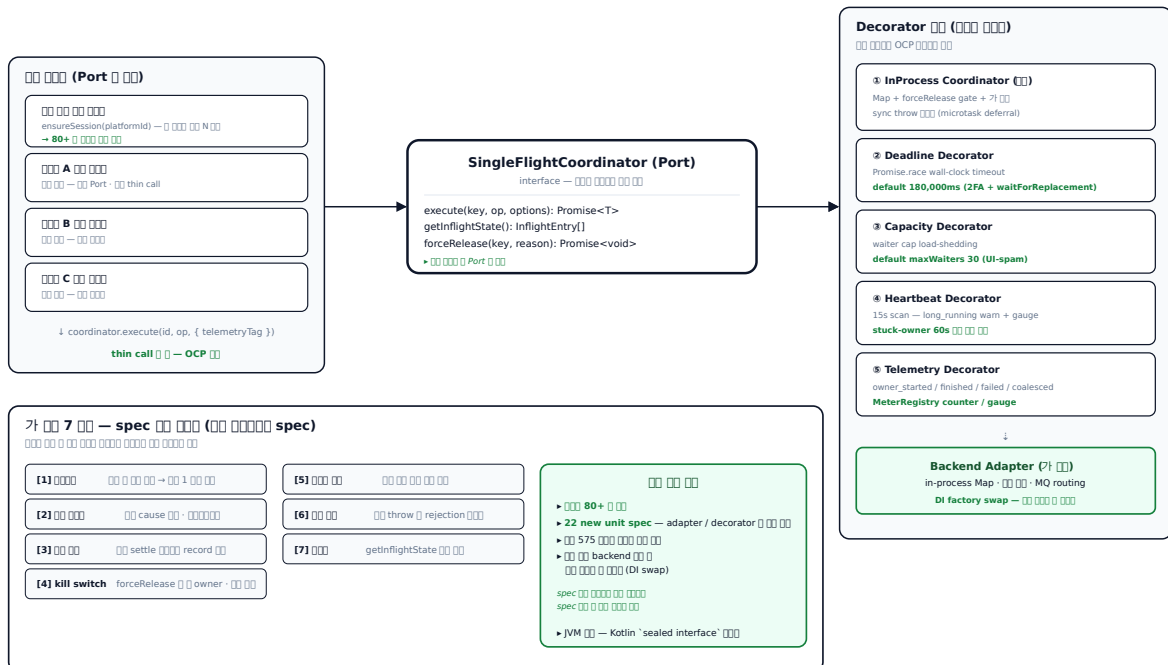
Port-Adapter(Hexagonal Architecture, Alistair Cockburn) + Decorator +

Strategy OCP + spec 7 describe 1:1 .

Port execute / getInflightState / forceRelease , in-process Map(forceRelease gate) + 4 (Deadline = Promise.race wall-clock timeout / Capacity = waiter cap load-shedding / Heartbeat = 15s scan long_running warn + gauge / Telemetry = owner_started/finished/failed/coalesced metrics) . 가 this.inflight.get(key) == record , sync throw Promise.resolve().then(() => operation()) hazard . default deadline 180,000ms(2FA + waitForReplacement), maxWaiters 30(UI-spam load-shedding), long_running warn at 60s(stuck-owner early signal), forceRelease ops kill-switch.

C4. Single-Flight Coordinator — Hexagonal Architecture + Decorator 5

가 7 spec — service 80+ → thin call



C4. Single-Flight Coordinator — Hexagonal + Decorator 5

가 7 — spec (cause / throw / getInflightState / forceRelease / owner ·)

#		
---	--	--

1	□□□□	□□ □ □□ □□□ 1□□ □□
2	□□ □□□	□□ cause □□□□ □□ (□□ □□□□ □□)
3	□□ □□	□□ settle □□□□ record □□ □□
4	kill switch	forceRelease □□ □□ □□□ □ owner, □□ □□
5	□□□ □□	□□ □□ □□ □□ □□
6	□□ □□	□□ throw □ promise rejection □□ □□□
7	□□□	getInflightState 가 entry □□ □□

□□

- □□□ □□ **80+** □ □□ — coordination + telemetry 가 thin call coordinator.execute(id, op, { telemetryTag }) □ □□.
- **22 new unit specs** — adapter/decorator □ □□ □□, □□ □□□ □□□ □□ □□.
- □□ □□ backend 가 □ □□□ □□ □ □□□□ DI factory □ swap.
- 「 □□□□□ spec □□ □□□□ □□□□ □□ □□□ □□ □□□□□□ □□ □□□ □□□.

JVM/Spring □□ — Spring @Component + @Qualifier □ backend □□□ swap · Kotlin Coroutines + Deferred □ in-process single-flight · Resilience4j Bulkhead.semaphore + TimeLimiter □ deadline · MeterRegistry counter / gauge □ telemetry.

14. SafeCron — **cron** **(14)** **2025.09 ~**

— @Cron **N** → PG API rate-limit, cron **N** → SaaS rate-limit + . cron **N** → SaaS rate-limit + . cron **N** → SaaS rate-limit + .

— **DI** + Slack + . ModuleRef lazy . TTL OOM / SIGKILL cron . Slack — start / complete (\${duration}ms) / error - \${message} + trace block — 가 . slackAlert: false cron , slackWebhookUrl cron . CRON_ENABLED noop → QA .

- **14** (subscriptions, SaaS, magic-link, mixpanel, log-cleanup).
- 5~10 → **100** + cron 가 .
- cron — QA .
- DI + Lock acquire/release/TTL + Slack + . 5~10 .

JVM/Spring — Spring @Scheduled + ShedLock (Redis/JDBC backend) — cron . @ConditionalOnProperty . Spring AOP @Around Slack .

15. 00 00 00 000000 — 00 → AI → 4 TOCTOU 0000 → API Key/JWT 00 00 00 00 2026.03

00 — 「0000 0 000 AI 가 」 00000 4 가:

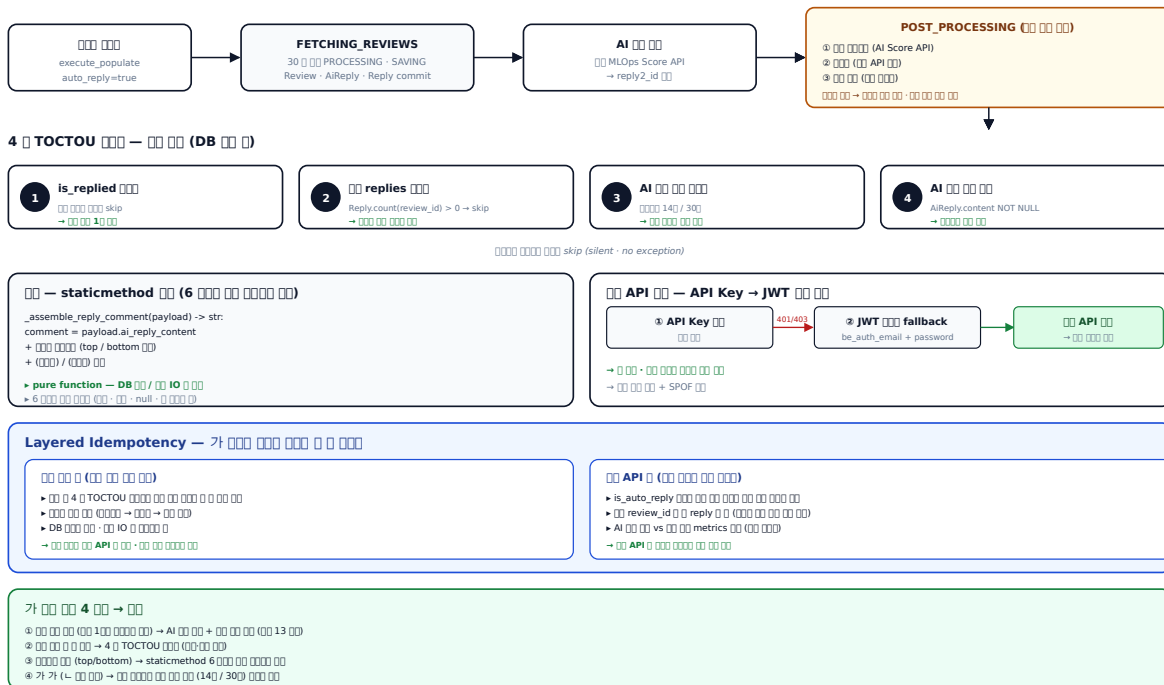
- 00 00 00 — 00 100 "0000 0000 00" 00 = 00
- 00 00 — 00 0000 0 0 00
- 0000 00 00 — 0000 00 0000 AI 00 0 / 00 00
- 00 0000 00 — 00000 50000 / 「40 00000 00

가 DB 00 0000 600 00000 0000 00 API 00 0 0000 00 0000 000000.

00 — 00000 00 00 + **API Key → JWT** 00 00 **fallback** + **TOCTOU 40** 0000 0 30 0000. payload 0000 00 DB 00, 0000 staticmethod, 0000 async 0 30 0000 00 00 0000 00000 DB 0000 00 0 0(Spring @Transactional 00 0000 00 00). 40 TOCTOU 00000 ① is_replied 0000 ② 00 replies 0000 ③ AI 00 00 0000 ④ AI 00 00 00 — 00000 00000 0000 skip. 0000 staticmethod 0 0000 6가 0000 00 00000 00. 00 API 0 00 API Key 00 → 401/403 00 0 JWT 000000 00, 0 00 · 00 0000 0000 00 00. 0000 00 00 00 — 00 00000 → 0000 → 00 00(가 0000, 0000 0000 0000 00 00000 00 0000 00 0000 00 00).

D1. 00 00 00 000000 — 00 → AI → 4 TOCTOU 0000 → 00 00 00

0000 00 00 + API Key → JWT fallback + Layered Idempotency 0 3 0 0000



D1. 00 00 00 000000 — TOCTOU 4 0000 + 00 00

Layered Idempotency 00 00

- 00 00 00: 00 0 40 00000 00 00 0 0 0 00 00(00 00 00 00)
- 00 API 00: 00000 0000 00 00 0000 00 00 0000 00(00 review_id 0 0 reply 0 0)
- 00 0000 00 **API** 0 00, 00 **API** 0 0000 00000 00 00 00 — 00000 0000 0000 0 0 00000 00 0000 00

- `Runnable` 이 `Callable` 으로 **AI** → `Callable` → `Future` 로 변경.
- `Runnable` 4개 가 `Runnable` / `Callable` 로 변경, `Runnable` `staticmethod` 6개 `Runnable` 로 변경.
- API Key → JWT fallback `Runnable` 이 `Runnable` 로 변경.
- AI `Runnable` vs `Runnable` `metrics` 가 「 — AI `Runnable` vs 「 」 로 변경.

JVM/Spring `Runnable` — Spring `@Transactional(readOnly=true)` `Runnable` + `RestClient` `Runnable` `Runnable`
`Runnable` · Spring Security `OAuth2AuthorizedClientManager` `Runnable` API Key → JWT fallback ·
 Resilience4j Retry + CircuitBreaker `Runnable`.

JVM 애플리케이션 — Node.js 애플리케이션 Kotlin/Spring 애플리케이션 + ADR

5개 애플리케이션 Node.js 애플리케이션 애플리케이션 애플리케이션 애플리케이션 애플리케이션 애플리케이션 애플리케이션 애플리케이션 애플리케이션 애플리케이션 Java/Kotlin/Spring 애플리케이션 애플리케이션 애플리케이션 애플리케이션 애플리케이션, 애플리케이션 애플리케이션 ADR 애플리케이션 애플리케이션 애플리케이션. 애플리케이션 **ADR 49**, 애플리케이션 **47**, 애플리케이션 애플리케이션 **17** 애플리케이션.

Node 애플리케이션 애플리케이션 ↔ Kotlin/Spring 애플리케이션 애플리케이션

Node 애플리케이션	Kotlin/Spring 애플리케이션
애플리케이션 애플리케이션 SETNX + Lua 애플리케이션 애플리케이션	Redisson watchdog + Pub/Sub 4개 애플리케이션
Idempotency Key + 애플리케이션 + 애플리케이션 + reconciliation	ADR-006 애플리케이션 애플리케이션 4개 + EXP-09b 9 애플리케이션
애플리케이션 애플리케이션 FIFO + lease TTL	Coroutines + supervisorScope + Single-Flight 5 invariants
Custom Error Strategy 5개 + 9개 애플리케이션	Resilience4j 5개 + 9개 애플리케이션 애플리케이션
애플리케이션 애플리케이션 PID Registry (OOM 애플리케이션)	JVM Heap / GC 애플리케이션 G1GC Region + Humongous
애플리케이션 애플리케이션 classic queue (애플리케이션 5개 애플리케이션 애플리케이션)	Kafka 애플리케이션 애플리케이션 + Outbox Relay

애플리케이션 애플리케이션

- 애플리케이션·애플리케이션 애플리케이션 애플리케이션 (Java / Spring Boot / JPA) — 애플리케이션 애플리케이션 4개 애플리케이션, 애플리케이션 애플리케이션 **4개** 애플리케이션(애플리케이션 · 애플리케이션 · GET_LOCK · Redisson), DB InnoDB RR vs ANSI RR 애플리케이션 애플리케이션 애플리케이션, 애플리케이션 애플리케이션 애플리케이션 9 애플리케이션 애플리케이션, No-offset Pagination.
- 애플리케이션 애플리케이션 애플리케이션 애플리케이션 (Spring Batch + Kafka) — 애플리케이션 애플리케이션 1개 애플리케이션 5개 애플리케이션(replay 애플리케이션, prefetch HoL, x-overflow, publisher confirm 애플리케이션, DLQ 애플리케이션 애플리케이션) 애플리케이션 애플리케이션 Kafka 애플리케이션 애플리케이션 ADR 애플리케이션. Outbox Relay 애플리케이션 애플리케이션 polling vs CDC 애플리케이션 애플리케이션, Spring Batch Reader 4개 애플리케이션.
- 애플리케이션 애플리케이션 애플리케이션 (Kotlin / Coroutines / Resilience4j) — 애플리케이션 애플리케이션 Kotlin 애플리케이션 애플리케이션. **Single-Flight** 애플리케이션 애플리케이션 애플리케이션(promise sharing, sync throw normalization, force release, deadline, capacity), 9 애플리케이션 애플리케이션 애플리케이션 애플리케이션, Resilience4j 5 애플리케이션(CircuitBreaker, Retry, Bulkhead, RateLimiter, TimeLimiter) 애플리케이션.

📄 — 프로젝트(I-BRICKS) | | 📅 2021.05 ~ 2024.11

📄 프로젝트는 📄 프로젝트 팀·원, 📄 프로젝트, 📄 프로젝트.

- 📄 프로젝트 팀·원 — Elasticsearch + Logstash 📄 RDB → ES 📄 프로젝트, 📄 프로젝트 팀·원 프로젝트가 📄 프로젝트 팀·원 프로젝트 📄 프로젝트 프로젝트.
- **EBS** 📄 프로젝트 프로젝트 — 📄 프로젝트 📄 프로젝트 Kafka + Apache Nifi + Elasticsearch 📄 프로젝트 프로젝트. 📄 프로젝트 프로젝트 **50%** 프로젝트, 📄 프로젝트 2가 📄 프로젝트.
- 📄 프로젝트 프로젝트 — React / Redux / SCSS 📄 프로젝트 UI, KWCAG 2.1 📄 프로젝트 프로젝트.

00 — 00000 0000 00 00 00

00 50 0000 0000 00 0000 0000000 000000.

00·0000

- 00 9 00000 00000 — 00 / 00 / 00·0000 / 00(00·00) / 00 / 00000 00 / 0000 00(00·00) / 00000 → 00 0000 00.
- 0000 00 00 0000 00 — 0000 DB 00 00 0 + SELECT FOR UPDATE 00 00 + 00 00 Pub/Sub 00 00 + 00 0000 00 00000 **TPS 100 → 1,000+**, 00 00 **50 → 100** 00, 00 0000 **1%** 00 00.

0 · 0000 · 0000

- 0000 0 DLX 00 application-level retry — setTimeout + republishMessage 0 hot loop 00, 30s / 5m / 30m 0000 backoff. Phase 2 00 DLX 0000 00 00000000 00 00.
- 00 00 IntegrityError-aware upsert + Lock-timeout 00 0000 — Duplicate 0 break, Lock timeout 0 1s / 2s / 3s 00 backoff, 00 00 raw SQL ON DUPLICATE KEY UPDATE 00.

00 0000 · 00 00

- 00 API quarantine — DB-backed state machine(OK / QUARANTINED) + 12h probe 00 00 + DB↔00 00 mismatch 00.
- 00 00000 00 00 + 00 00 00000 — 00 00(00 0000 0000 ++) / 00 API 00 200, 00 30 00 00000 + 00 0 0000 00 00 00.
- 0000 00 00 0000 — 00000 180 + 110+ 0000 BasePlatformException → 00000 0000 → classifier helper 0 3 layer 0. instanceof + name + code 30 0000 0000 00 00.
- Idempotent Reply Pipeline — Lease + Completion Cache 00 + 40 ErrorClassifier(BLOCKED_SUSPECTED / TRANSIENT / PERMANENT / SYSTEM) + ACK/NACK 00000 0000. 0000 PERMANENT(unknown 00 00 retry 00).
- Adaptive Traffic Controller + Launch Budget — Lua Token Bucket + Circuit Breaker(SOFT_OPEN / HALF_OPEN) + jittered watchdog 00 Thundering Herd 00.

00 00 · 0000

- 00000 00000 0 — Launch Semaphore + Prewarm Pool + Quarantine + 5-phase Zombie Cleanup + Jittered Watchdog 50 00 00. 40 concurrent 00 **97.2%** 0000.
- OOM 00 chunk 00 — 00 00 00 cron 0 find 0000 0000 CHUNK_SIZE=2000 while 00 + lookup map 10 00000 00.
- 0000 00000 0000 — 0000 00000 300 00 + Semaphore 30 + 00 DB 00 + 0000 gc + 300 00 0000000 00 0 0000. 0000 00000 00 gc 0000 0000.

00000 · 00

- 00 00000 000000000 — 00 000000000 00 00 Hash + ZSet(0000 / 0000 0000) + TTL 0000 00 (RUNNING 6h / COMPLETED 1h / FAILED 2h)0 00. 00 0000 0 orphaned job 00 0000 (Semaphore 10 0000 00).
- 2400 00 0000 + Config 0 RUNNING 10 — 2400 0 0000 DB 0 00 00 ±50 0000 dedupe 0 000 00, Config 0 RUNNING 10 00 + 00 00 Semaphore(200).

- `backfill` — `fetch in-memory set` `backfill`, `backfill` `backfill` `backfill`.
- `backfill` `backfill` `backfill`.
- `backfill` `backfill` + `backfill` SaaS `backfill` `backfill` — `backfill` `backfill` → `backfill` `backfill` `backfill` `backfill` `backfill`, `backfill` `backfill` + 5 `retry queue` `backfill` `backfill`.
- APM Span Tagging — `tagDatadogSpan` `backfill` `backfill` `backfill`(`backfill` `backfill`, `backfill` amount mismatch) `backfill` `backfill`.

☐☐ ☐☐

☐☐	☐☐ ☐☐
☐☐	TypeScript, JavaScript, Kotlin , Java , Python
☐☐☐☐☐	NestJS, Node.js, Spring Boot , Spring Batch , FastAPI
ORM	TypeORM, Python ORM, JPA / Hibernate , APScheduler
☐☐☐☐☐☐	☐☐☐ DB (Cluster, Replica) — MySQL ☐☐ / PostgreSQL / Oracle
☐☐ · ☐	☐☐ ☐☐ (Cluster ☐☐), Lua eval, Token Bucket, Redisson
☐☐☐	☐☐☐ ☐ (Quorum Queue, DLX, classic queue ☐☐ ☐ ☐), Kafka
☐☐	Elasticsearch, Logstash, Apache Nifi
☐☐ ☐☐☐ ☐☐	Resilience4j , Custom Retry / Timeout, Single-Flight (Port-Adapter), Circuit Breaker (SOFT_OPEN / HALF_OPEN), Token Bucket
☐☐☐☐ ☐☐☐☐	stealth fork ☐☐ ☐☐☐☐ ☐, ☐☐ ☐☐ 5☐
☐ ☐☐ ☐☐	Akamai · Cloudflare ☐☐ — ☐☐ ☐☐ 4-state ☐☐, ☐☐ ☐☐ polling, Referrer Warming, Click Loop jitter
☐☐ · ☐☐	Prometheus, Grafana, APM (span tagging) , nGrinder, JFR
☐☐☐	Jest, Testcontainers , JUnit 5 , Kotest , Pytest, "☐☐ spec ☐☐ ☐☐☐" ☐☐☐
☐☐☐	Docker, Docker Compose × 8 color, ☐☐ ☐☐☐ ☐☐, ☐ ☐☐☐ ☐☐☐☐☐☐☐, ☐☐ Cloud, Jenkins, GitHub Actions
☐☐☐☐☐☐	Flyway, Liquibase ☐☐
☐☐	Slack Webhook (Block Kit), ☐☐☐ OAuth2 + ☐☐☐ ☐☐ ☐☐☐
AI / ML	☐☐ MLOps Score API + 6☐☐ ☐☐☐ ☐☐ (Shannon entropy, zlib ☐☐☐, n-gram, emoji run) + 가 ☐ ☐☐ (Bell + Polarity Amplification + tanh box)
☐☐ ☐☐☐ ☐☐	Redlock(Martin Kleppmann) ABA ☐☐ fencing token, Hexagonal Architecture(Port-Adapter), CQRS, Saga, Outbox, CAS, Bulkhead, Circuit Breaker, Single-Flight



F-Lab Java Backend Mentoring | 2024.01 ~ 2024.07

Meta ☐ ☐ ☐ ☐ ☐ ☐. ☐ ☐ ☐, ☐ ☐ ☐, ☐ ☐ ☐ ☐ ☐ ☐, CQRS, ☐ ☐ ☐ ☐ ☐.

☐ ☐ ☐ ☐ ☐ ☐